CampusGraph

Student Network & Collaboration Analyzer

Student: Mary Vaasha Micfran
Roll Number: 2025BCS289
Department: CSE

1. Abstract

CampusGraph is a Python-based student network analysis system that models relationships between students using a graph. The project uses Pandas for data handling, NetworkX for graph construction and analysis, SQLite for database integration, Matplotlib for network visualization, and Streamlit for an interactive dashboard. The system identifies highly connected students, detects communities, finds shortest relationships, and recommends potential new collaborators based on shared skills.

2. Problem Statement

In a college environment, students are connected through courses, projects, skills and other interactions. These relationships can be difficult to understand using ordinary tables alone. CampusGraph represents student interactions as a network so that important connections, groups and possible collaborations can be identified systematically.

3. Objectives

• Represent students and their interactions as a graph.
• Analyze student connectivity using degree and degree centrality.
• Identify communities within the student network.
• Find shortest paths between students.
• Identify isolated students.
• Recommend new collaborators using common skills.
• Store and query project data using SQLite.
• Provide an interactive Streamlit dashboard.

4. Dataset

The project uses a synthetic/anonymized campus dataset. The notebook contains 20 students, 80 skill records, 60 course records, 8 projects, 20 student-project relationships and 36 student interaction records.

Dataset	Records
Students	20
Skills	80
Courses	60
Projects	8
Student-Project relationships	20
Interactions	36

5. Methodology

Data Loading: CSV files are loaded using Pandas. **Graph Construction:** each student is a node and each recorded interaction is an edge. **Network Analysis:** degree and degree centrality are calculated using NetworkX. **Community Detection:** greedy modularity community detection identifies groups. **Shortest Path:** NetworkX finds the minimum number of connections between students. **Recommendation:** directly connected students are excluded and remaining students are ranked by shared skills. **Database:** datasets are stored in SQLite and queried using SQL through Pandas. **Dashboard:** Streamlit provides student selection, statistics, recommendations and graph visualization.

6. Implementation

The main implementation is contained in CampusGraph.ipynb. The interactive interface is implemented in app.py. The project uses Python 3.14.5. The notebook successfully builds a 20-node, 36-edge student graph, performs centrality analysis, detects communities, computes a shortest path, generates collaborator recommendations, stores datasets in SQLite and creates a network visualization.

7. Results

Metric	Result
Total students	20
Student connections	36
Communities	4
Isolated students	0
Top centrality	Anu and Diya — 0.263
Shortest path example	Anu → Diya — 1 connection
Mary's top skill-match score	4

For Mary (S001), the recommendation system identified Vishnu and Devika as the highest skill-match candidates, with four shared skills each. Meera, Akhil and Joel followed with three shared skills each.

8. Database Integration

SQLite is used to store the six datasets as database tables: students, skills, courses, projects, student_projects and interactions. SQL queries were executed through Pandas. The project queried Year 3 students and counted students by year; the result was 11 Year 2 students and 9 Year 3 students.

9. Streamlit Dashboard

The Streamlit dashboard provides a user-friendly interface for selecting a student and viewing their number of connections, degree centrality, skills and recommended new collaborators. It also displays the overall student network graph and summary statistics. The dashboard was successfully tested locally using Streamlit.

10. System Output and Dashboard

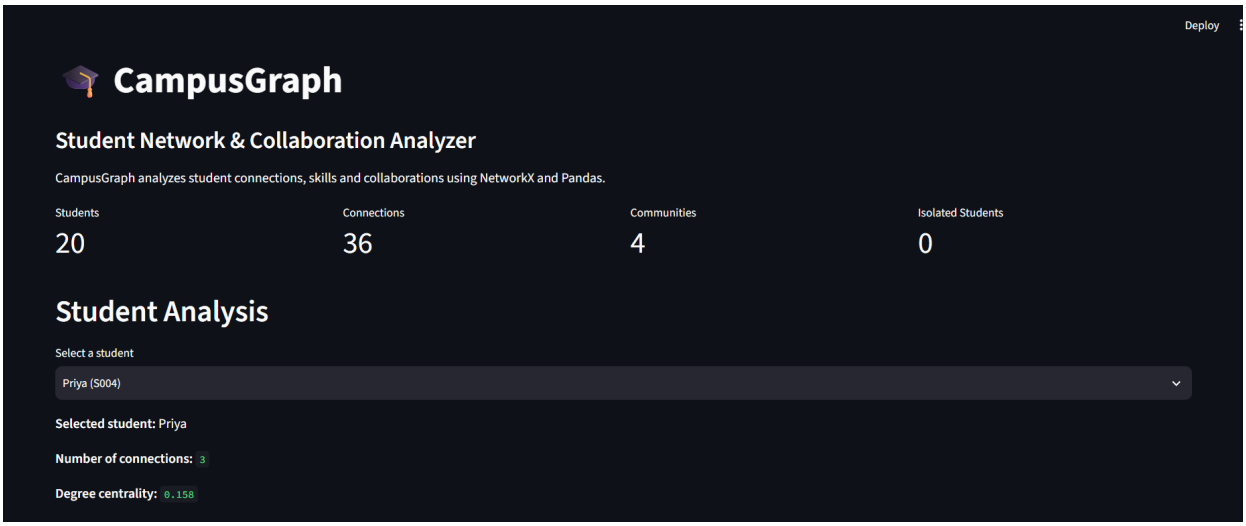


Figure 1. CampusGraph dashboard showing overall statistics and student analysis.

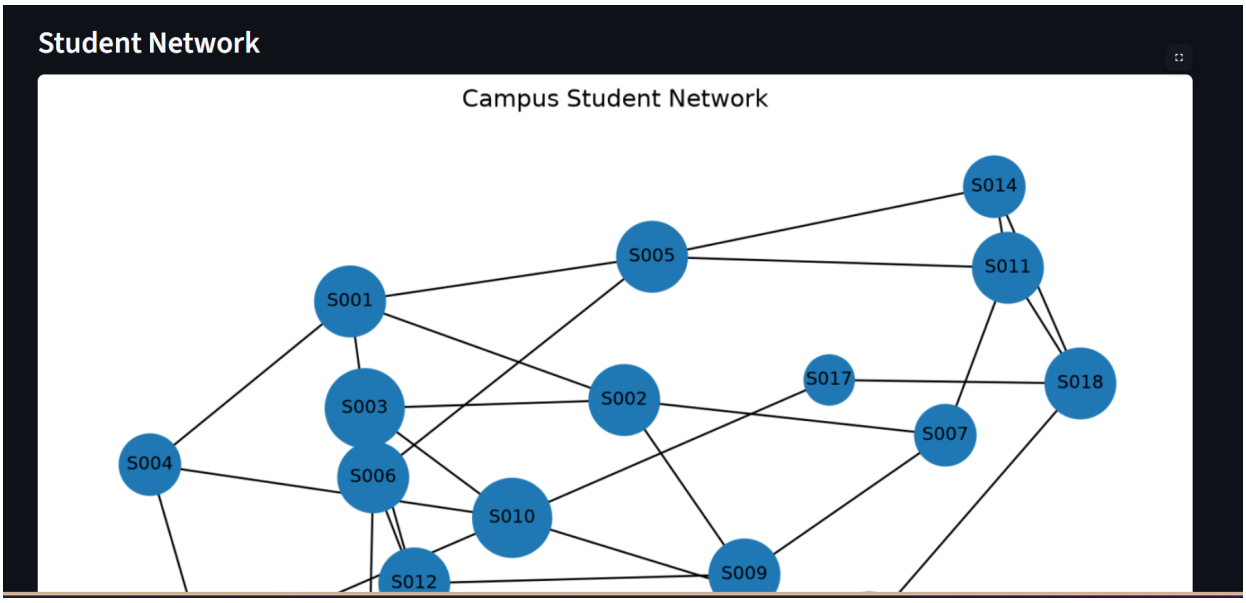


Figure 2. Student network visualization generated using NetworkX.

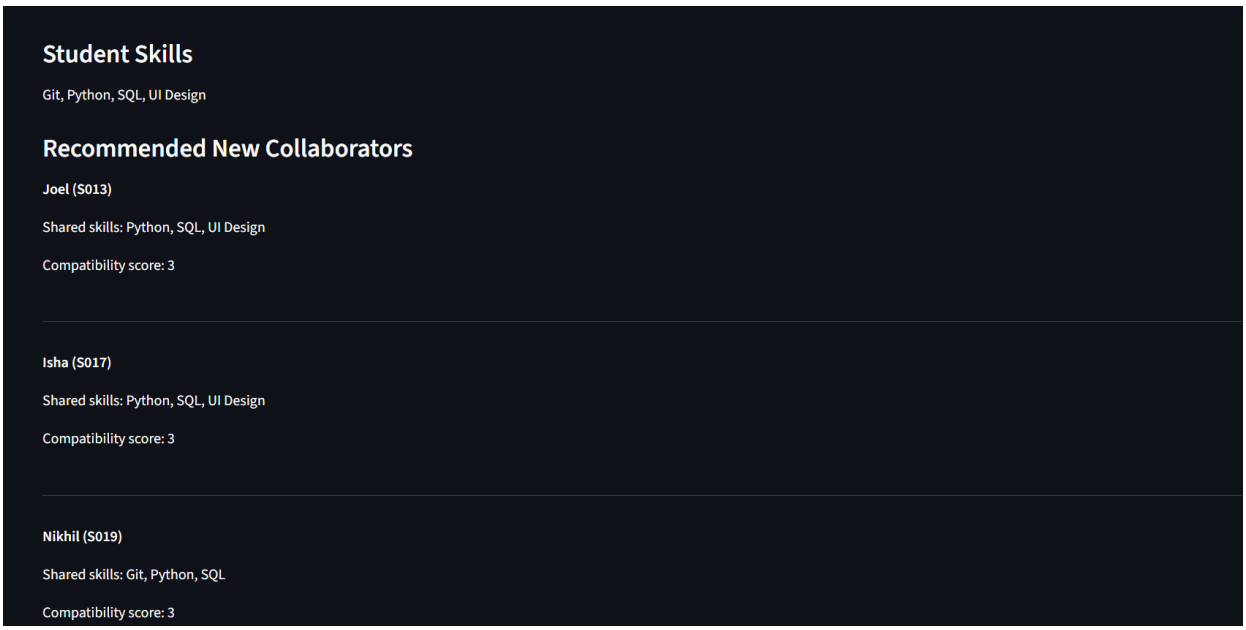


Figure 3. Collaborator recommendations based on shared skills.

11. Conclusion

CampusGraph demonstrates how graph-based analysis can be applied to a student network. The system successfully combines data analytics, NetworkX graph algorithms, database integration and a Streamlit user interface. The results show how central students, communities, relationships and potential collaborators can be identified from structured student data.

12. Future Enhancements

• Allow users to upload real campus datasets.
• Add more centrality measures such as betweenness and closeness centrality.
• Add interactive community visualization.
• Include course and project filters in the dashboard.
• Use weighted interactions to represent stronger relationships.
• Add authentication and role-based access for real deployments.

13. Technologies Used

Python 3.14.5, Pandas, NetworkX, Matplotlib, SQLite, Streamlit, Jupyter Notebook and CSV.